

1η Εργασία στα Νευρωνικά Δίκτυα Βαθιά Μάθηση

Φλοριάν Μπυλυκμπάσι
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
ΑΕΜ:11012

November 23, 2025

Εισαγωγή - Γενικές Πληροφορίες

Για την ενδιάμεση εργασία χρησιμοποιήσαμε Visual Studio Code όπου μας ζητούσαν η χρήση δύο machine learning αλγορίθμων του K-NearestNeighbor και του NearestCentroid για την κατηγοριοποίηση ενός dataset της επιλογής μας. Για να επιτευχθεί αυτό ήταν αναγκαία η δημιουργία ενός περιβάλλοντος .venv ώστε να μπορούν να εγκατασταθούν οι κατάλληλες βιβλιοθήκες μέσω pip install.

Για την κύρια εργασία χρησιμοποιήσαμε Google Colab επειδή ήταν αναγκαία η χρήση της T4 GPU που δίνει η Google καθώς τα μοντέλα μας θέλανε αρκετή ώρα για να εκπαιδευτούν. Για την εργασία αυτή έχουν υλοποιηθεί σχεδόν 10 διαφορετικά μοντέλα στα οποία κάθε φορά αλλάζουμε κάτι ώστε να δούμε πως θα επηρεαστεί το δίκτυο μας (ποσο σωστά θα εκπαιδευτεί, τι ακρίβεια θα έχει). Στο τέλος υπάρχουν τρία παραδείγματα κατηγοριοποίησης.

Οι βιβλιοθήκες που έχουν χρησιμοποιηθεί είναι οι tensorflow, numpy , time , sklearn και matplotlib.pyplot.

Από τα δοσμένα dataset επιλέχτηκε η Cifar10 η οποία είναι μια συλλογή από 60.000 32x32 εικόνες χωρισμένες σε 10 διαφορετικές κλάσεις. Οι κλάσεις αυτές είναι:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

Τέλος, είναι χωρισμένη σε ένα training set απο 50.000 εικόνες και σε ένα testing set από 10.000 εικόνες. Για τους σκοπούς της εργασίας χρησιμοποιούμε ένα **validation set** 10.000 εικόνων και μόνο 40.000 εικόνες από το training set.

Παρακάτω ξεκινάει η ανάλυση πρώτα των δυο κατηγοριοποιητών και έπειτα ανάλυση του κάθε μοντέλου ξεχωριστά. Στο τέλος θα υπάρχουν σχόλια και συμπεράσματα για τα αποτελέσματα μας.

1-NearestNeighbor

Ο κατηγοριοποιητής αυτός για $K=1$ πιάνει μια εικόνα την οποία θέλει να ταξινομήσει. Υπολογίζει λοιπόν από αυτήν την ευκλείδια απόσταση της από όλες τις εικόνες στο training set μας. Από αυτές τις αποστάσεις μετά κρατάει τη μία που είναι πιο κοντά στην δικιά μας εικόνα. Έτσι μετά η εικόνα μας παίρνει την κλάση της πιο κοντινής εικόνας (π.χ. αν η πιο κοντινή εικόνα είχε κλάση 'cat' τότε και η δικιά μας θα πάρει την ίδια).

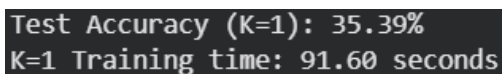
Στον κώδικα μας στην αρχή κάνουμε load το dataset μας μέσω της tensorflow. Έπειτα έχουμε την επιλογή είτε να χρησιμοποιήσουμε όλο το dataset μας είτε μόνο ένα κομμάτι αυτού. Στο παραδειγμά μας χρησιμοποιούμε όλο το dataset αλλά ο χρήστης έχει την επιλογή να μειώσει τα δείγματα ώστε να δει τις αλλαγές στην ακρίβεια. Επειδή ο αλγόριθμος δεν δέχεται 2D εικόνες κάνουμε flatten τις εικόνες σε ένα μεγάλο διάνυσμα 3072 στοιχείων ($32 \times 32 \times 3$). Έπειτα μέσω των βοηθητικών εντολών των βιβλιοθηκών μας γίνεται η εκπαίδευση και η αξιολόγηση και στο τέλος εκτυπώνονται τα αποτελέσματα.

3-NearestNeighbor

Παρόμοια με πριν ο κατηγοριοποιητής πιάνει μία εικόνα και υπολογίζει την ευκλείδια απόσταση της από όλες τις υπόλοιπες. Τώρα η διαφορά είναι ότι κρατάει τις 3 πιο κοντινές αποστάσεις και κοιτάζει τις κλάσεις αυτών των εικόνων. Στο τέλος θα επιλέξει την πλειοψηφία αυτών των κλάσεων (π.χ. αν έχω 2 κλάσεις 'cat' και 1 'frog', επιλέγει 'cat').

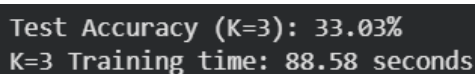
Ο κώδικας για $K=3$ βρίσκεται στον ίδιο κώδικα με αυτόν για $K=1$ απλώς προσθέσαμε την εκπαίδευση και την αξιολόγηση στις βοηθητικές εντολές της βιβλιοθήκης για $K=3$ τώρα και στο τέλος τυπώνουμε τα αποτελέσματα.

Αποτελέσματα για NearestNeighbor



```
Test Accuracy (K=1): 35.39%
K=1 Training time: 91.60 seconds
```

Figure 1: Αποτελέσματα για $K=1$



```
Test Accuracy (K=3): 33.03%
K=3 Training time: 88.58 seconds
```

Figure 2: Αποτελέσματα για $K=3$

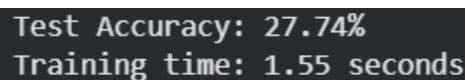
Παρατηρούμε ότι και τα δύο έχουν παρόμοιους χρόνους εκπαίδευσης , γύρω στα 90 δευτερόλεπτα. Ωστόσο για $K=1$ έχουμε κατά 2% μεγαλύτερη ακρίβεια από ότι για $K=3$. Οπότε μπορούμε εύκολα να συμπεράνουμε ότι για ολόκληρο το dataset για $K=1$ ο αλγόριθμος δουλεύει καλύτερα από ότι για $K=3$.

Nearest Centroid

Η βασική ιδέα τώρα εδώ είναι ότι κάθε κλάση πλέον αντιπροσωπεύεται από το κέντρο των δειγμάτων της στον χώρο των χαρακτηριστικών. Στην αρχή λοιπόν ο αλγόριθμος υπολογίζει το μέσο όρο όλων των χαρακτηριστικών των δειγμάτων. Όταν λοιπόν έρθει ένα νέο δείγμα τότε υπολογίζουμε την απόσταση του από όλα αυτά τα κέντρα που έχουμε. Το δείγμα μετά θα πάρει την κλάση της μικρότερης απόστασης.

Ο κώδικας χρησιμοποιεί ακριβώς την ίδια λογική με πριν , το μόνο που αλλάζει πλέον είναι ότι χρησιμοποιούνται οι βοηθητικές εντολές της βιβλιοθήκης για την NearestCentroid από ότι την NearestNeighbor για την εκπαίδευση και την αξιολόγηση.

Αποτελέσματα για NearestCentroid



Test Accuracy: 27.74%
Training time: 1.55 seconds

Figure 3: Αποτελέσματα για NearestCentroid

Βλέπουμε εδώ ότι ο χρόνος εκπαίδευσης είναι εξαιρετικά μικρός ωστόσο η ακρίβεια είναι αρκετά πιο χαμηλή σε σχέση με τον NearestNeighbor (7% πιο κάτω).

Συμπεράσματα-Σχόλια

Μπορούμε εύκολα να συμπεράνουμε αντικειμενικά ότι ο καλύτερος απο τους 3 είναι ο 1-NearestNeighbor καθώς είναι πιο αποδοτικός και ας έχει ελάχιστα μεγαλύτερο χρόνο απο τον 3-NearestNeighbor. Ο πιο γρήγορος είναι ο NearestCentroid , ωστόσο λόγω της κακής του απόδοσης καθίσταται τελευταίο της λίστας. Ο 3-NearestNeighbor λόγω της χαμηλότερης κατά 2% απόδοση , βρίσκεται κάτω απο τον 1- NearestNeighbor.

Υλοποίηση Νευρωνικού Δικτύου για την Cifar10

Στο σημείο αυτό ακολουθεί η κύρια ιδέα της εργασίας που είναι η υλοποίηση ενός νευρωνικού δικτύου εμπρόσθιας τροφοδότησης το οποίο θα μπορούσε να ήταν είτε MLP (πλήρως συνδεδεμένο) είτε CNN (συνελικτικό) είτε συνδυασμός των δύο. Παρακάτω υπάρχουν υλοποιήσεις 2 μοντέλων MLP και αρκετών μοντέλων συνδυασμών. Πριν από αυτά όμως θα αναφερθούν κάποιες γενικές πληροφορίες για το κάθε νευρωνικό δίκτυο και το πως δουλεύει το καθένα. Επίσης θα σχολιαστούν και θα αναλυθούν κάποιοι σημαντικοί όροι οι οποίοι είναι αναγκαίοι για την κατανόηση των μοντέλων μας.

Multi Layer Perceptrons

Όταν λέμε εμπρόσθια τροφοδότηση εννοούμε ότι η πληροφορία κινείται προς την μια κατεύθυνση από το επίπεδο εισόδου, στα όσα κρυφά επίπεδα υπάρχουν και τέλος στο επίπεδο εξόδου χωρίς να υπάρχει κάποια επανάληψη ή ανάδραση.

Τα MLP είναι ένα είδος ενός νευρωνικού δικτύου εμπρόσθιας τροφοδότησης τα οποία αποτελούνται από πλήρως συνδεδεμένα πυκνά επίπεδα τα οποία μετατρέπουν δεδομένα από μια διάσταση σε μια άλλη. Λέγονται πολύ-επίπεδα επειδή περιέχουν ένα επίπεδο εισόδου, κανένα, ένα, ή πολλά κρυφά επίπεδα και ένα επίπεδο εξόδου.

Στο κρυφό επίπεδο ο κάθε νευρώνας υπολογίζει το άθροισμα των βαρών της εισόδου συν ακόμα ένα bias όρο. Έπειτα αυτό το άθροισμα περνάει μέσα από μία συνάρτηση ενεργοποίησης ώστε να εισάγει την μη γραμμικότητα που θέλουμε. **Στα παραδείγματα μας χρησιμοποιούμε την ReLU ως συνάρτηση ενεργοποίησης.** Αφού το δίκτυο παράξει μια έξοδο τότε ακολουθεί ο υπολογισμός του σφάλματος μέσω της χρήσης κάποιας συνάρτησης σφάλματος. Στα παραδείγματα μας χρησιμοποιείται η Sparse Categorical Crossentropy.

Μετά απο αυτά ακολουθεί ο backpropagation αλγόριθμος ο οποίος αυτό που κάνει είναι να ελαχιστοποιεί την διαφορά μεταξύ της προβλεπόμενης και της πραγματικής εξόδου. Ο τρόπος που δουλεύει είναι να παίρνει τα σφάλματα προς τα πίσω στο δίκτυο, χρησιμοποιώντας τον κανόνα αλυσίδας του λογισμού που υπολογίζει τα ανάδελτα, και μετά αναδρομικά ενημερώνει τα βάρη και τα biases.

Τέλος αυτά τα δίκτυα βασίζονται πάρα πολύ σε αλγορίθμους βελτιστοποίησης τα οποία αναδρομικά βελτιώνουν τα βάρη και τα biases την ώρα της εκπαίδευσης.

Convolutional Neural Networks

Τα συνελικτικά νευρωνικά δίκτυα έχουν έναν λίγο πιο περίπλοκο σχεδιασμό. Αποτελούνται και αυτά πάλι από ένα επίπεδο εισόδου αλλά τώρα το κρυφό επίπεδο αποτελείται από ένα συνελικτικό επίπεδο, από ένα pooling επίπεδο και ένα πλήρως συνδεδεμένο επίπεδο (μέσα σε αυτά υπάρχει και το επίπεδο ενεργοποίησης). Στο τέλος κλασικά υπάρχει το γνωστό όπως πριν επίπεδο εξόδου.

Στην αρχή το επίπεδο εισόδου είναι αυτό που όπως πριν δίνουμε την είσοδο στο μοντέλο μας το οποίο κρατάει τις διαστάσεις όπως είναι.

Έπειτα ακολουθεί πρώτα το συνελικτικό επίπεδο στο οποίο προσπαθούμε να εξάγουμε την μορφή της εικόνας. Σε αυτό το επίπεδο εφαρμόζονται ένα σετ απο φίλτρα στις εικόνες μας τα οποία συνήθως είναι μικροί πίνακες. Αυτό που εξάγεται από το επίπεδο αυτό είναι οι διαστάσεις που είχαμε πριν με μονή αλλαγή την τελευταία διάσταση στην οποία τώρα θα μπει ο αριθμός των φίλτρων.

Μετά έχουμε ένα επίπεδο ενεργοποίησης στο οποίο παρόμοια με τα MLP εισάγουμε την συνάρτηση ενεργοποίησης στην έξοδο του προηγούμενου επιπέδου. **Και εδώ γίνεται χρήση της ReLU στα παραδείγματα μας.**

Ύστερα έχουμε το pooling επίπεδο το οποίο η κύρια λειτουργία του είναι να μειώσει τις δύο πρώτες διαστάσεις μας κάνοντας έτσι τους υπολογισμούς γρηγορότερος ενώ ταυτόχρονα μειώνει την μνήμη και μπορούμε να αποφύγουμε την υπερεκπαίδευση. Εμείς κάνουμε χρήση του max pooling.

Πριν περάσει η πληροφορία στο πλήρως συνδεδεμένο επίπεδο κάνουμε flatten την πληροφορία σε 1D διάνυσμα καθώς είναι ο μόνος τρόπος ώστε να εισέλθει στο επίπεδο αυτό. Εκεί κάνει τον τελευταίο απαραίτητο υπολογισμό κατηγοριοποίησης πριν περάσει στο επίπεδο εξόδου.

Στο επίπεδο εξόδου χρησιμοποιούμε την συνάρτηση softmax η οποία μετατρέπει την έξοδο αυτή που πήραμε στο πιθανοτικό αποτέλεσμα κάθε κλάσης.

Οι αλγόριθμοι βελτιστοποίησης καθώς και ο αλγόριθμος backpropagation έχουν ακριβώς την ίδια λογική με αυτήν στα MLP.

Παρακάτω ακολουθεί η ανάλυση βασικών ορών καθώς είναι πράγματα τα οποία χρησιμοποιούνται αρκετά στα μοντέλα μας.

Βασικοί Όροι

- **Overfit(Υπερεκπαίδευση):** Συμβαίνει όταν το μοντέλο μαθαίνει πάρα πολλά από τα δεδομένα εκπαίδευσης. Έτσι δούλευε καλά το μοντέλο σε αυτά τα δεδομένα όχι όμως όταν δοκιμάζεται σε νέα δεδομένα. Συνήθως συμβαίνει όταν το μοντέλο είναι πολύπλοκο ή λόγω του μεγέθους των δεδομένων εκπαίδευσης.
- **Underfit(Υποεκπαίδευση):** Εδώ έχουμε το αντίθετο του overfit, διότι το μοντέλο δεν μπορεί να καταλάβει τι γίνεται με τα δεδομένα. Έτσι το μοντέλο δεν δουλεύει καλά ούτε στην εκπαίδευση ούτε στην δοκιμή. Λόγοι μπορεί να είναι η απλότητα του μοντέλου, ανεπαρκές μέγεθος των δεδομένων εκπαίδευσης, ή υπερβολική χρήση τεχνικών κανονικοποίησης που βοηθούν στην αποφυγή υπερεκπαίδευσης.
- **Epoch(Εποχές):** Είναι το πόσες φορές το μοντέλο σου θα επεργάζεται τα δεδομένα σου. Στα παραδείγματα κυρίως χρησιμοποιούμε 100.
- **Batch size:** Πόσα δείγματα θα επεξεργαστούν πριν ενημερωθεί το μοντέλο μας. Στα παραδείγματα χρησιμοποιούμε κυρίως 64.
- **Validation data(Δεδομένα Επικύρωσης):** Είναι ένα κομμάτι από δεδομένα διαφορετικά από το βασικό training set τα οποία χρησιμοποιούνται ώστε να δούμε πόσο καλά γενικεύεται το μοντέλο σε νέα δεδομένα ώστε να ρυθμίσουμε κατάλληλα τις παραμέτρους εκπαίδευσης.
- **Early Stopping(Πρώρη Διακοπή):** Είναι μια τεχνική η οποία σταματά την εκπαίδευση πριν ολοκληρωθούν όλα τα epochs επειδή το μοντέλο δεν βελτιώνεται άλλο ή αρχίζει να χειροτερεύει λόγω overfit.
- **Learning Rate(Ρυθμός Μάθησης):** Είναι το πόσο μεγάλα βήματα κάνει το δίκτυο μας όταν ενημερώνει τα βάρη κατά την εκπαίδευση. Όταν είναι μεγάλος ο ρυθμός το μοντέλο μαθαίνει γρήγορα όμως μπορεί να γίνει ασταθές ενώ όταν είναι μικρό το μοντέλο μαθαίνει πιο αργά αλλά σταθερά.
- **Dropout:** Είναι τεχνική που απενεργοποιεί τυχαία ένα ποσοστό νευρώνων βοηθώντας έτσι να αποτρέψουμε την υπερεκπαίδευση. Το μοντέλο δηλαδή εκπαιδεύεται από συγκεκριμένους νευρώνες.
- **Batch Normalization:** Τεχνική που ομαλοποιεί(κανονικοποιεί) τις τιμές των νευρώνων μέσα σε κάθε batch κατά την εκπαίδευση. Με αυτήν την τεχνική σταθεροποιείται η εκπαίδευση, επιτρέπεται η χρήση μεγαλύτερου learning rate και αυξάνεται η ταχύτητα εκπαίδευσης.
- **Weight decay ή L2 Regularization:** Είναι και αυτή τεχνική που προσθέτει μια μικρή ποινή στα μεγάλα βάρη του μοντέλου ώστε να μην μεγαλώσουν υπερβολικά. Στόχος είναι η μείωση υπερεκπαίδευσης και η σταθεροποίηση του μοντέλου.

Αυτοί ήταν κυρίως οι όροι οι οποίοι χρήζαν ανάλυσης καθώς στη συνέχεια χρησιμοποιούνται συνέχεια στα μοντέλα μας. Επίσης θα βοηθήσει πολύ αυτή η ανάλυση των όρων και στην ανάλυση του κώδικα που ακολουθεί μετά.

Ανάλυση Κώδικα

Ξεκινάμε κάνοντας import όλες τις απαραίτητες βιβλιοθήκες που χρησιμοποιούνται μετά στον κώδικα . Πολλά απο αυτά που υπάρχουν στον κώδικα μας έχουν ήδη αναφερθεί ήδη πάνω.

Κάνουμε load την cifar10 και κανονικοποιούμε απευθείας τα pixel και κάνουμε flatten τις ετικέτες ώστε να είναι η διαδικασία πιο εύκολη. Έπειτα χωρίζουμε τα δεδομένα μας σε training set, validation set και test set. Το training set έχει 40.000, το validation έχει 10.000 και το test μένει όπως ήταν με 10.000 εικόνες.

Μετά ξεκινάει η κατασκευή του μοντέλου μας, όπου παίρνουμε αρχικά τον αριθμό των κλάσεων μας. Έπειτα προστίθενται τα διάφορα επίπεδα μας αναλόγως με το τι έχουμε στο μυαλό μας να κατασκευάσουμε. Η λογική της κατασκευής αυτής βασίζεται στις πληροφορίες που δώσαμε πριν για τα δύο είδη νευρωνικών δικτύων που μας δόθηκαν. Γενικά τα παραδείγματα ξεκινάνε από απλά μοντέλα και όσο προχωράμε γίνονται πιο πολύπλοκα.

Στην συνέχεια κάνουμε compile το μοντέλο μας χρησιμοποιώντας τον Adam Optimizer και την Sparse Categorical Crossentropy για όλα τα παραδείγματα μας. Επίσης έχουμε χρήση της Early Stopping αν παρατηρήσουμε ότι δεν γίνεται βελτίωση για 25 εποχές ενώ παράλληλα αποθηκεύουμε τα καλύτερα βάρη. Μετά απο αυτά έχουμε προσθέσει το ReduceLROnPlateau το οποίο αν δει ότι το μοντέλο χειροτερεύει , χαμηλώνει λίγο το learning rate ώστε να σταθεροποιήσει κάπως το μοντέλο μας.

Αφού περάσουμε από όλα αυτά φτάνουμε στο σημείο που ξεκινάει η εκπαίδευση του μοντέλου μας με τις εποχές και το batch size που αναφέραμε πριν. Επίσης υπολογίζουμε MONO μια φορά την ακρίβεια και το σφάλμα σε ολόκληρο το test set.

Στο τέλος τυπώνουμε όλες τις απαραίτητες πληροφορίες που ζητάει η άσκηση και 2 επιπρόσθετα γραφήματα που δείχνουν την σχέση μεταξύ της ακρίβειας εκπαίδευσης με την ακρίβεια επικύρωσης και την σχέση μεταξύ σφάλματος εκπαίδευσης με το σφάλμα επικύρωσης αντίστοιχα.

Όσο περνάνε τα παραδείγματα θα δούμε στον κώδικα ότι προστίθενται οι διάφορες τεχνικές κανονικοποίησης που αναλύθηκαν πριν. Ας δούμε τώρα αναλυτικά ένα-ένα τα μοντέλα μας.

Παράδειγμα-Μοντέλο 1

- Σε αυτό το παράδειγμα κάνουμε χρήση MLP με δύο κρυφά επίπεδα στα οποία βάζουμε απο 64 νευρώνες στο καθένα.
- Δεν κάνουμε ΚΑΜΙΑ χρήση κάποιας τεχνικής κανονικοποίησης ώστε να βοηθήσουμε το μοντέλο μας, μόνο του ReduceLROnPlateau που η λειτουργία του εξηγήθηκε πιο πάνω.
- Έχουμε 100 εποχές , 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 212.87 seconds"
- "Test accuracy: 0.4802 , Test loss: 1.4668"

και έχουμε τα εξής διαγράμματα:

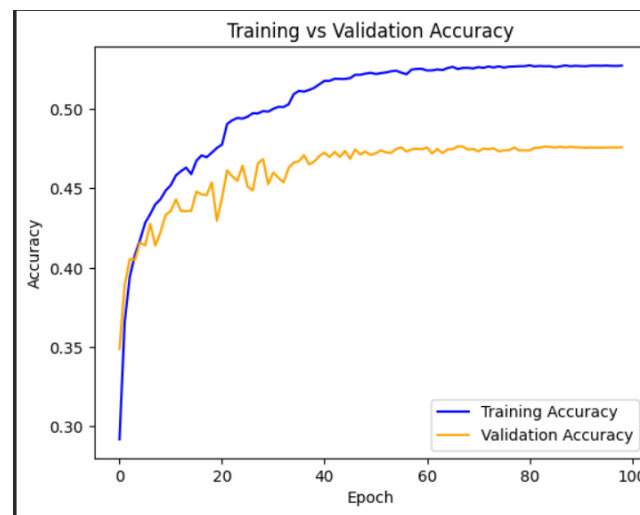


Figure 4: Διάγραμμα ακρίβειας 1ου παραδείγματος

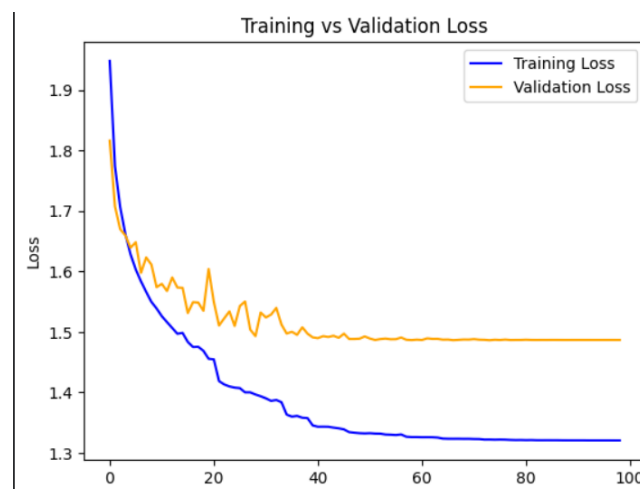


Figure 5: Διάγραμμα σφάλματος 1ου παραδείγματος

Βλέπουμε ότι το μοντέλο απο το διάγραμμα της ακρίβειας στις πρώτες εποχές εκπαιδεύεται ραγδαία που σημαίνει ότι μαθαίνει γρήγορα τα βασικά χαρακτηριστικά και μοτίβα των δεδομένων. Έπειτα οι δυο καμπύλες ανεβαίνουν σταδιακά σχεδόν μαζί έως τις 40 εποχές. Απο εκεί και μέχρι τις 100 εποχές παρατηρείται ένα κενό ανάμεσα στις δύο καμπύλες ενώ μάλιστα σταθεροποιούνται και οι δύο σε κάποιες τιμές(και στα δυο διαγράμματα μας) , γεγονός που δηλώνει ότι απο εκεί και πέρα το μοντέλο μας κάνει overfit.

Η ακρίβεια είναι σχετικά μικρή (κάτω απο 50%) διότι το μοντέλο μας είναι σχετικά απλό και αποτελείται απο σχετικά μικρό αριθμό νευρώνων.

Παράδειγμα-Μοντέλο 2

- Σε αυτό το παράδειγμα κάνουμε χρήση MLP με δύο κρυφά επίπεδα στα οποία βάζουμε απο 32 νευρώνες στο καθένα αυτή τη φορά.
- Δεν κάνουμε πολλές χρήσεις τεχνικών κανονικοποίησης , μόνο του Dropout(30%) και του ReduceLROnPlateau .
- Έχουμε 100 εποχές , 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 191.44 seconds"
- "Test accuracy: 0.3400 , Test loss: 1.7964"

και έχουμε τα εξής διαγράμματα:

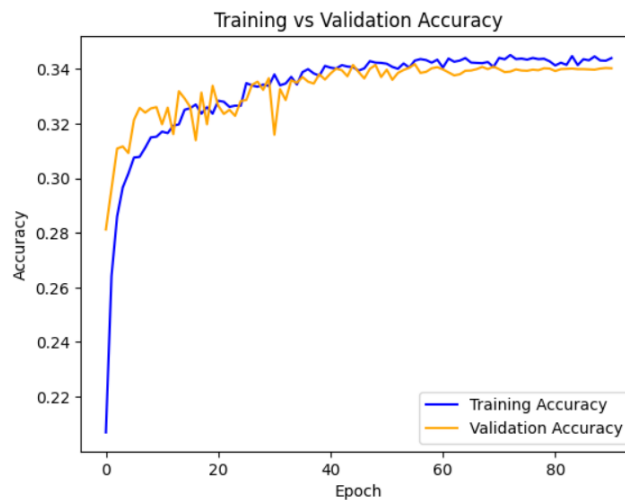


Figure 6: Διάγραμμα ακρίβειας 2ου παραδείγματος

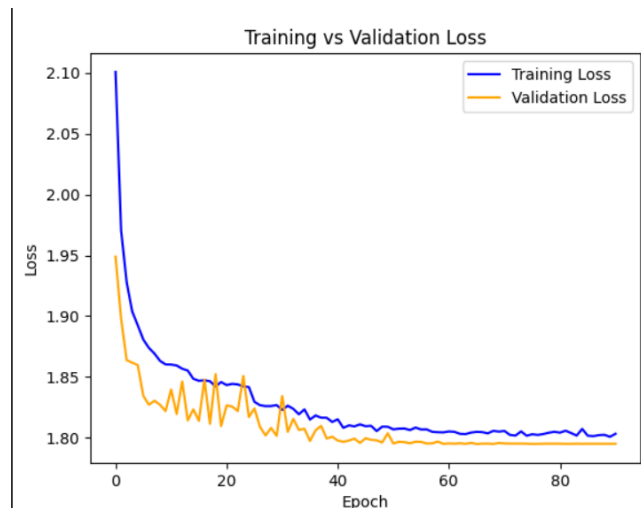


Figure 7: Διάγραμμα σφάλματος 2ου παραδείγματος

Πάλι εδώ βλέπουμε στις 10 πρώτες εποχές μια ραγδαία αύξηση στην ακρίβεια στις καμπύλες μας δείχνοντας ότι το μοντέλο μας μαθαίνει γρήγορα τα βασικά χαρακτηριστικά. Μετά οι δύο καμπύλες μας φαίνεται ότι ανεβαίνουν με ένα πιο αργό και σταθερό ρυθμό και σε αντίθεση με πριν ανεβαίνουν σχετικά μαζί, δηλαδή οι καμπύλες είναι κοντά η μια στην άλλη. Όμοια και αντίθετα τα ίδια συμβαίνουν και στις καμπύλες σφάλματος. Δεν έχουμε λοιπόν overfit, αλλά αντίθετα το μοντέλο μαθαίνει πολύ αποδοτικά. Στο τέλος στις τελευταίες εποχές αρχίζουν και σταθεροποιούνται οι καμπύλες της ακρίβειας γύρω στο 0.34 και του σφάλματος κοντά στο 1.80.

Έχουμε πιο μικρή ακρίβεια από πριν επειδή χρησιμοποιούμε λιγότερους νευρώνες και επίσης η Dropout τεχνική απενεργοποιεί παράλληλα κάποιους από αυτούς τους νευρώνες μας.

Παρακάτω θα δούμε μοντέλα πιο πολύπλοκα που θα αυξήσουν κατά πολύ την ακρίβεια ωστόσο θα προκαλέσουν σημαντικά overfit. Καθώς προχωράμε θα προσθέτουμε layers και θα βρούμε τρόπους να σταματήσουμε αυτό το πρόβλημα του overfit κρατώντας την ακρίβεια ψηλά.

Παράδειγμα-Μοντέλο 3

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 2 επιπέδων Conv2D (2 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και μετά από όλα αυτά το MaxPooling επίπεδο. Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 32 νευρώνων
- Δεν κάνουμε χρήσεις τεχνικών κανονικοποίησης, μόνο του ReduceLROnPlateau.
- Έχουμε 100 εποχές, 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 114.41 seconds"
- "Test accuracy: 0.6525, Test loss: 1.0197"

και έχουμε τα εξής διαγράμματα:

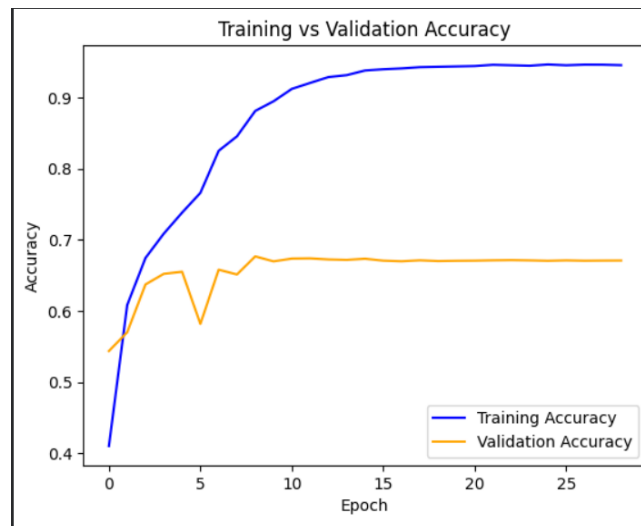


Figure 8: Διάγραμμα ακρίβειας 3ου παραδείγματος

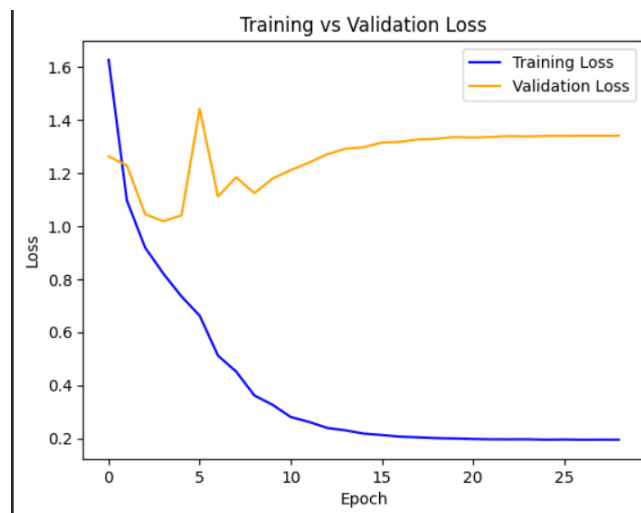


Figure 9: Διάγραμμα σφάλματος 3ου παραδείγματος

Ξεκινάμε πάλι στην ακρίβεια με ραγδαία αύξηση απο το 0.4 στο 0.55 περίπου στις πολύ πρώτες εποχές. Η καμπύλη του training πηγαίνει εξαιρετικά γρήγορα γύρω στο 0.9 ενώ η validation καμπύλη σιγά και σταθερά στο 0.65. Παρατηρούμε ένα τεράστιο κενό μεταξύ των καμπυλών (είτε ακρίβειας είτε σφάλματος) πράγμα που δηλώνει σοβαρό overfit. Το μοντέλο παύει να εκπαιδεύεται και κάπου στις 30 εποχές καλείται η Early Stopping.

Αυτό που καταφέραμε ωστόσο είναι σε σχέση με πριν είναι να αυξήσουμε την ακρίβεια κατά ένα πολύ σημαντικό ποσοστό (σχεδόν κατά 20%).

Παράδειγμα-Μοντέλο 4

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 4 επιπέδων Conv2D (4 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 2 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 128 νευρώνων
- Δεν κάνουμε χρήσεις τεχνικών κανονικοποίησης , μόνο του ReduceLROnPlateau .
- Έχουμε 100 εποχές , 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 168.45 seconds"
- "Test accuracy: 0.7609 , Test loss: 0.8037"

και έχουμε τα εξής διαγράμματα:

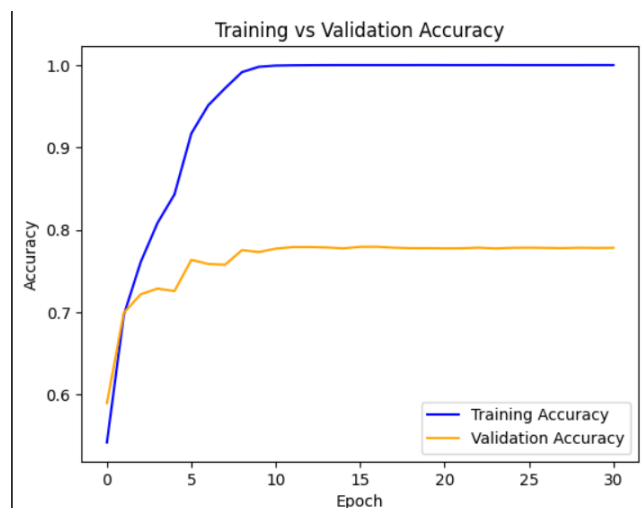


Figure 10: Διάγραμμα ακρίβειας 4ου παραδείγματος

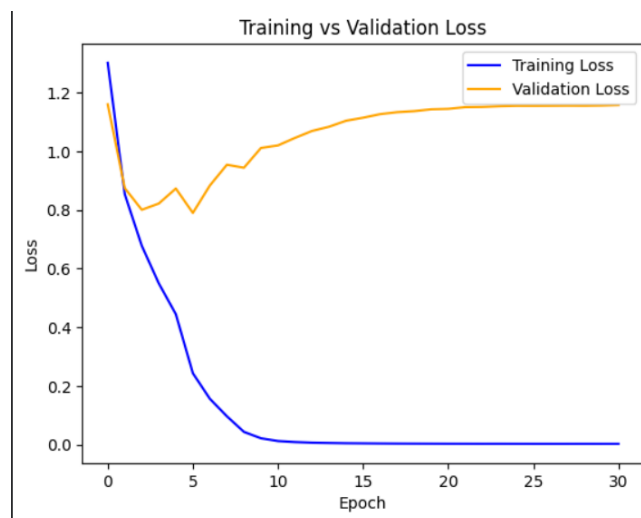


Figure 11: Διάγραμμα σφάλματος 4ου παραδείγματος

Στην αρχή ισχύει το ίδιο με όλα τα προηγούμενα παραδείγματα ωστόσο αυτή τη φορά πηγαίνουμε απο το 0.55 στο 0.7 περίπου. Η καμπύλη του training αρχίζει και αποκλίνει παρα πολύ απότομα ήδη απο τις πρώτες εποχές και καταφέρνει και φτάνει μεχρι και 1.0 (πλήρης απομνημόνευση). Η άλλη καμπύλη σταθεροποιείται κάπου στο 0.77 οπότε έχουμε πάλι αρκετό overfit και για αυτό καλείται η Early Stopping κάπου στις 30 εποχές.

Καταφέραμε ωστόσο με την αύξηση ενός μπλοκ να κερδίσουμε πάλι αρκετά στην ακρίβεια κατα ενα ποσοστό περίπου 10%.

Παράδειγμα-Μοντέλο 5

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 6 επιπέδων Conv2D (6 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 3 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ενα Fully Connected Layer 256 νευρώνων
- Δεν κάνουμε χρήσεις τεχνικών κανονικοποίησης , μόνο του ReduceLROnPlateau .
- Έχουμε 100 εποχές , 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 190.41 seconds"
- "Test accuracy: 0.7682 , Test loss: 0.6907"

και έχουμε τα εξής διαγράμματα:

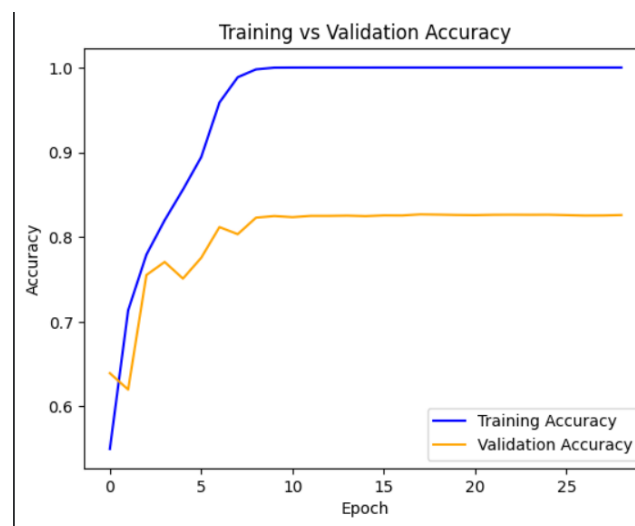


Figure 12: Διάγραμμα ακρίβειας 5ου παραδείγματος

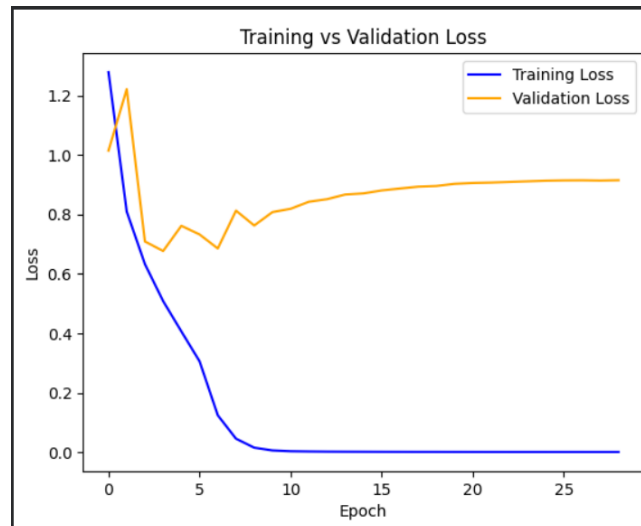


Figure 13: Διάγραμμα σφάλματος 5ου παραδείγματος

Η κατάσταση εδώ είναι ακριβώς πανομοιότυπη με πριν οπότε δεν έχουμε πολλά να εξηγήσουμε. Η ακρίβεια αυξήθηκε κατά ελάχιστα και το σφάλμα κατάφερε να πέσει κατα 0.1. Συμπεραίνουμε λοιπόν ότι τα 6 συνελκτικά επίπεδα αυτά (τα 3 μπλοκ μαζί δηλαδή με όλα τα άλλα επίπεδα) είναι το "όριο" για το μοντέλο και πως δεν χρειάζεται να το κάνουμε πιο πολύπλοκο. Παρακάτω λοιπόν θα εισάγουμε τεχνικές που θα σταματήσουν το overfit κρατώντας και την ακρίβεια ωστόσο ψηλά.

Παράδειγμα-Μοντέλο 6

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 6 επιπέδων Conv2D (6 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 3 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 256 νευρώνων
- Εκτός του ReduceLROnPlateau, πλέον βάζουμε και ένα Dropout(75%) πριν την έξοδο.
- Έχουμε 100 εποχές , 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 233.17 seconds"
- "Test accuracy: 0.8100 , Test loss: 0.6199"

και έχουμε τα εξής διαγράμματα:

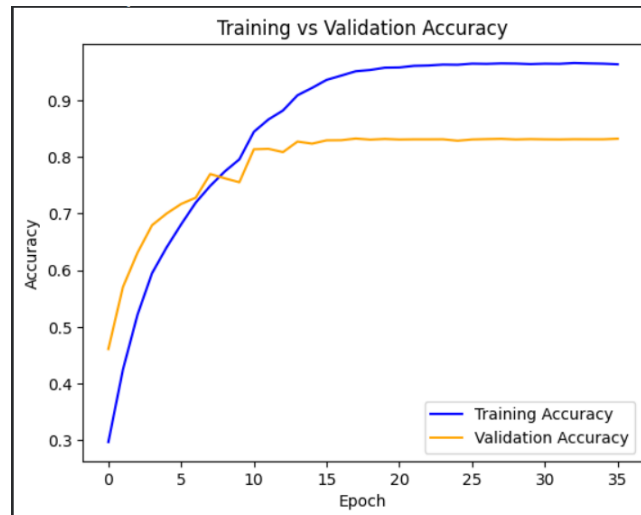


Figure 14: Διάγραμμα ακρίβειας του παραδείγματος

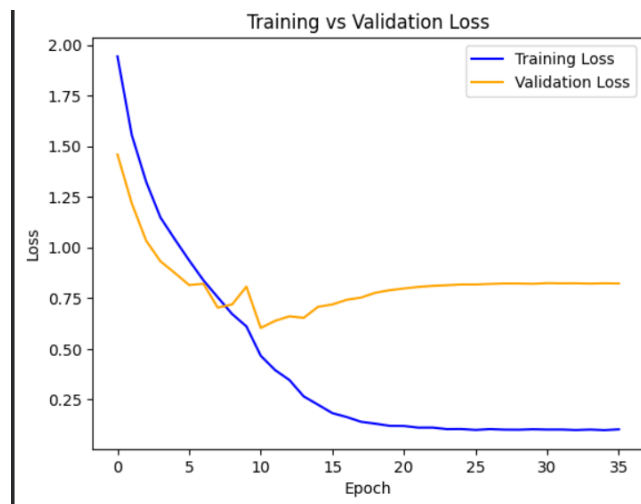


Figure 15: Διάγραμμα σφάλματος του παραδείγματος

Τα πράγματα εξακολουθούν να είναι τα ίδια στην αρχή και γενικά ως την 10η εποχή περίπου έχουμε ένα καλό ξεκίνημα με το μοντέλο να εκπαιδεύεται σωστά και να φτάνει ως και το 0.78 σχεδόν. Μετά υπάρχει μια σχετικά ελεγχόμενη απόκλιση σε σχέση με πριν περίπου 13% με την training καμπύλη να αγγίζει το 0.95 και την validation καμπύλη το 0.82. Όστόσο μετά υπάρχει μια σταθεροποίηση αναγκάζοντας έτσι την κλήση της Early Stopping ώστε να αποθηκευτούν τα καλύτερα βάρη.

Το μόνο σίγουρο είναι ότι αυτό το μοντέλο είναι σίγουρα καλύτερο από τα προηγούμενα καθώς καταφέραμε και αγγίξαμε ακρίβεια πάνω από 0.8. Ωστόσο το overfit ακόμα παραμένει που σημαίνει ότι πρέπει κάτι ακόμα να προσθέσουμε ώστε να βοηθήσουμε το μοντέλο μας.

Παράδειγμα-Μοντέλο 7

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 6 επιπέδων Conv2D (6 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 3 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 256 νευρώνων
- Πλέον χρησιμοποιούμε ReduceLROnPlateau, Dropout μετά από κάθε μπλοκ με διάφορες τιμές συν το Dropout που υπήρχε πριν (σύνολο δηλαδή 4 Dropout) και Weight Decay-L2 Regularization με $l2(1e-4)$.
- Έχουμε 100 εποχές, 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 553.50 seconds"
- "Test accuracy: 0.8631, Test loss: 0.6295"

και έχουμε τα εξής διαγράμματα:

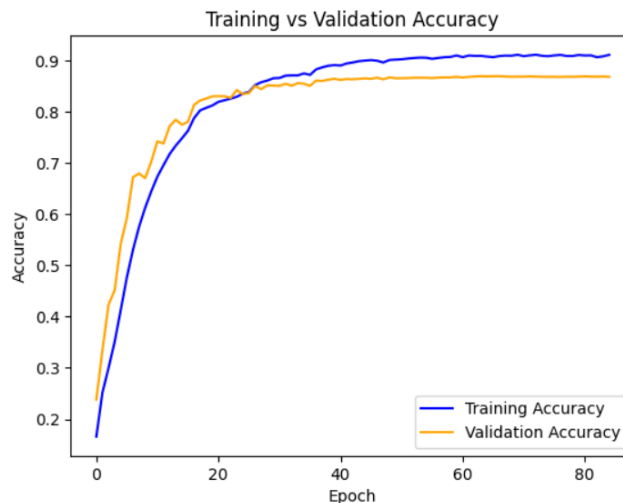


Figure 16: Διάγραμμα ακρίβειας 7ου παραδείγματος

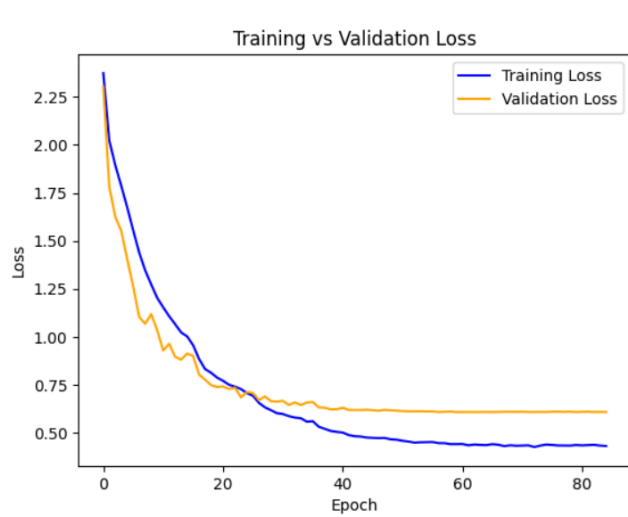


Figure 17: Διάγραμμα σφάλματος 7ου παραδείγματος

Σε αυτό το παράδειγμα έχουμε λοιπόν μια υγιή διαδικασία εκπαίδευσης σε σχέση με τα προηγούμενα πολύπλοκα μοντέλα μας. Οι καμπύλες της ακρίβειας στην αρχή πηγαίνουν στις πρώτες 30 εποχές έως και στο 0.8 μαζί (είναι δηλαδή η μία πολύ κοντά στην άλλη πράγμα που δηλώνει ότι το μοντέλο μαθαίνει πολύ αποτελεσματικά σε νέα δεδομένα). Έπειτα από εκεί και πέρα η καμπύλη του training σταθεροποιείται κάπου στο 0.9 και η validation στο 0.85 και αυτή η μικρή διαφορά θεωρείται φυσιολογική και αποδεκτή. Ωστόσο το μοντέλο μετά από λίγο παύει να βελτιώνεται και γύρω στις 85 εποχές πραγματοποιείται Early Stopping.

Η ακρίβεια που πήραμε είναι η μέγιστη από όλα τα παραδείγματα που φτάσαμε και αυτό οφείλεται κυρίως στις πολλές βοηθητικές τεχνικές κανονικοποίησης που χρησιμοποιήσαμε.

Στα επόμενα 2 παραδείγματα θα χρησιμοποιήσουμε μόνο ένα learning rate το οποίο όμως θα είναι αρκετά μικρότερο από το 0.001 που ξεκινάμε στα προηγούμενα παραδείγματα.

Παράδειγμα-Μοντέλο 8

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 6 επιπέδων Conv2D (6 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 3 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 256 νευρώνων
- Χρησιμοποιούμε ένα Dropout(75%) μόνο πάλι, Weight Decay-L2 Regularization με $l2(1e-4)$ και learning rate ίσο με $5e-5$ αυτή τη φορά.
- Έχουμε 100 εποχές, 64 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 369.74 seconds"
- "Test accuracy: 0.7253, Test loss: 0.9863"

και έχουμε τα εξής διαγράμματα:

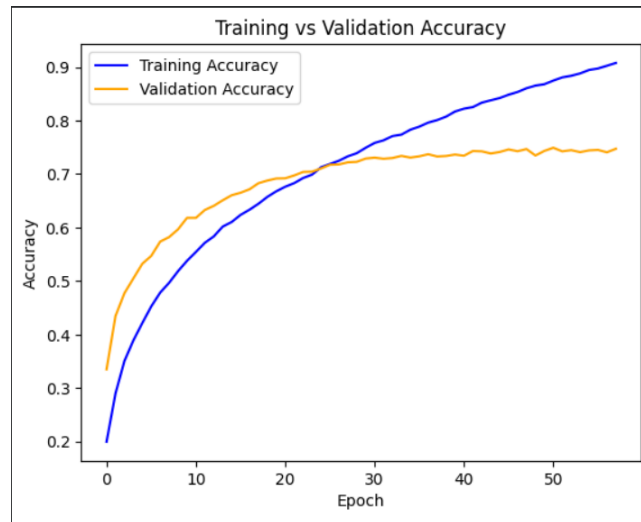


Figure 18: Διάγραμμα ακρίβειας δού παραδείγματος

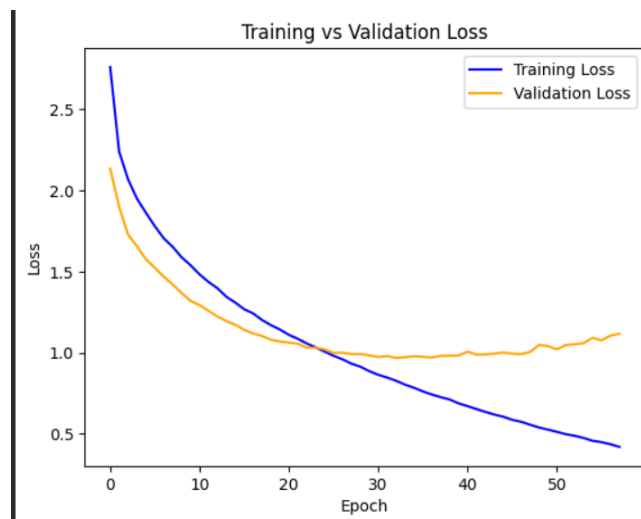


Figure 19: Διάγραμμα σφάλματος δού παραδείγματος

Βλέπουμε πως λόγω του μικρού ρυθμού εκμάθησης που ρυθμίσαμε η καμπύλη της επικύρωσης στην ακρίβεια είναι πιο πάνω στην αρχή από αυτή του training. Επίσης η χρήση του Dropout μειώνει το σκορ εκπαίδευσης και για αυτό είναι πιο κάτω. Μετά η καμπύλη training περνάει από πάνω και φτάνει ως και το 0.9 ενώ η άλλη μας καμπύλη βλέπουμε σταθεροποιείται γύρω στο 0.7. Έχουμε δηλαδή πάλι overfit.

Η ακρίβεια λογικό να είναι πιο κάτω αφού ο ρυθμός εκμάθησης τώρα είναι σχετικά χαμηλός.

Παράδειγμα-Μοντέλο 9

- Σε αυτό το παράδειγμα στο κρυφό επίπεδο κάνουμε χρήση 6 επιπέδων Conv2D (6 συνελκτικά επίπεδα), το καθένα να έχει ένα Batch Normalization επίπεδο και 3 MaxPooling επίπεδα (ένα ανα 2 Conv2D επίπεδα). Η έξοδος αυτών βρίσκει ένα Fully Connected Layer 256 νευρώνων
- Χρησιμοποιούμε ένα Dropout(75%) μόνο πάλι, Weight Decay-L2 Regularization με $l2(1e-3)$ και learning rate ίσο με $1e-5$ αυτή τη φορά.

- Έχουμε 150 εποχές , 32 batch size και Adam Optimizer.

Ο κώδικας μας τυπώνει:

- "Total training time = 833.35 seconds"
- "Test accuracy: 0.6884 , Test loss: 1.5575"

και έχουμε τα εξής διαγράμματα:

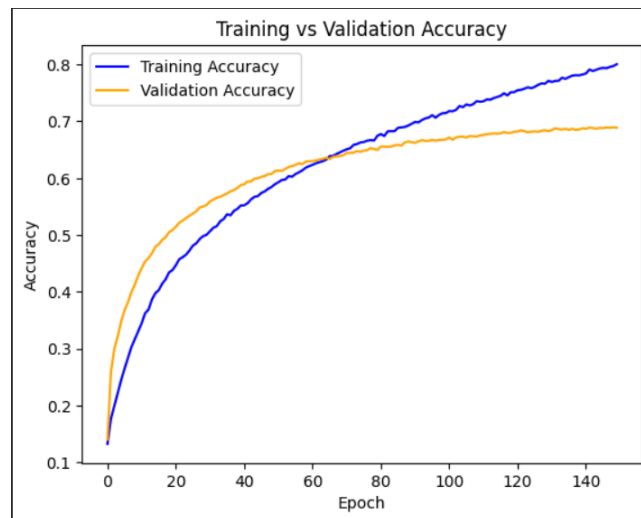


Figure 20: Διάγραμμα ακρίβειας 9ου παραδείγματος

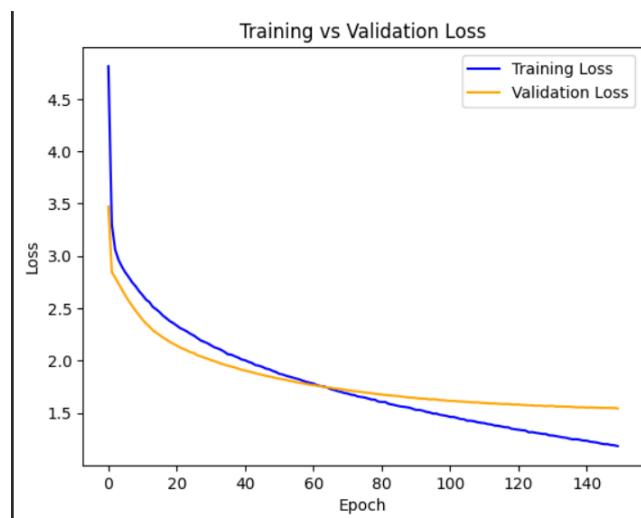


Figure 21: Διάγραμμα σφάλματος 9ου παραδείγματος

Σε αντίθεση με όλα τα προηγούμενα παραδείγματα εδώ παρατηρούμε το φαινόμενο του underfit. Αυτό παρατηρείται στο διάγραμμα ακρίβειας από το γεγονός ότι η καμπύλη της επικύρωσης είναι για 60 εποχές πάνω από την καμπύλη του training. Έπειτα η καμπύλη του training ξεπερνάει αυτή της επικύρωσης και συνεχίζει την ανοδική σταδιακή πορεία ενώ η άλλη σταθεροποιείται κάπου στο 0.65 χωρίς σημάδια βελτίωσης. Μετά εκεί στο τέλος μπορούμε να θεωρήσουμε ότι έχουμε μια ήπια μορφή overfit. Η ακρίβεια λογικό να είναι τόσο χαμηλή αφού μειώσαμε το learning rate σε υπερβολικό βαθμό ($1e-5$).

Παραδείγματα Κατηγοριοποίησης

Παρακάτω έχουμε τρία παραδείγματα κατηγοριοποίησης, τα δύο πρώτα αφορούν ένα αδύναμο μοντέλο, στα οποία στην αρχή προβλέπει σωστά και μετά όχι. Στο τρίτο ένα καλύτερο μοντέλο πιάνει την ίδια εικόνα και βλέπουμε να προβλέπει σωστά τώρα σε σχέση με πριν. Ο κώδικας κάνει load το μοντέλο που θέλουμε να δοκιμάσουμε, μετά εκτυπώνουμε την εικόνα που δοκιμάζουμε και ύστερα μέσω των συναρτήσεων της tensorflow βγάζουμε το αποτέλεσμα και το τυπώνουμε.



Figure 22: Ορθή κατηγοριοποίηση/2ο μοντέλο



Figure 23: Λανθασμένη κατηγοριοποίηση/2ο μοντέλο

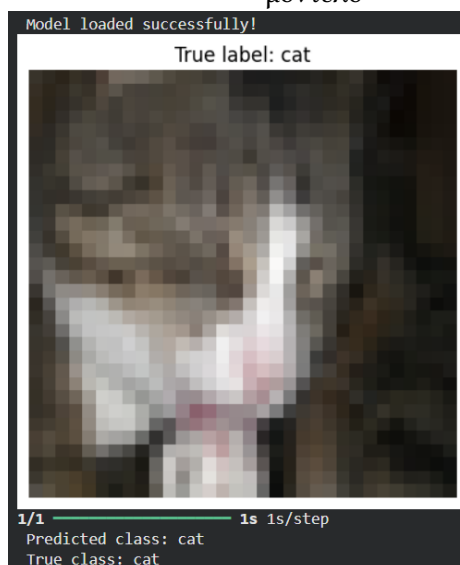


Figure 24: Ορθή κατηγοριοποίηση/7ο μοντέλο

Τελικά Σχόλια για την Ολική Εργασία

- Παρατηρούμε ότι τα MLP μοντέλα μας, παρουσιάζουν σχετικά ομαλή εκπαίδευση ωστόσο μικρή ακρίβεια.
- Όσο πιο απλό είναι το μοντέλο μας τόσο μικρότερη θα είναι η ακρίβεια.
- Για αυτό και είναι εφικτό να δούμε ακρίβεια πιο μικρή και από τους αλγορίθμους KNN και NC που χρησιμοποιήσαμε στην αρχή.
- Γενικά όμως η υλοποίηση ενός νευρωνικού δικτύου θεωρείται καλύτερη ειδικά όταν έχουμε πολύπλοκα δεδομένα όπως την Cifar10.
- Τα CNN μοντέλα μας αν και πετυχαίνουν μεγαλύτερη ακρίβεια (ειδικά όσο πιο πολύπλοκα γίνονται) παρουσιάζουν αρκετά το φαινόμενο του overfit.
- Καταλήξαμε ότι το όριο των συνελκτικών επιπέδων ήταν 6 (ή 3 μπλοκ) καθώς δεν παρατηρούσαμε παραπάνω βελτίωση.
- Ο σκοπός μετά ήταν με αυτά τα επίπεδα να λύσουμε αρκετά το φαινόμενο του overfit κρατώντας την ακρίβεια ψηλά πράγμα που καταφέραμε με τις διαφορές τεχνικές κανονικοποίησης.
- Το καλύτερο μας μοντέλο κατάφερε να κάνει υγιή εκπαίδευση και να φτάσει και 0.86 ακρίβεια.
- Τέλος είδαμε πως η χρήση πιο χαμηλού learning rate ενώ διώχνει το overfit μας ρίχνει χαμηλά και την ακρίβεια.